

繼承

継承の前に

オブジェクト指向にはSOLIDの原則という考え方があります。
まずはこの考え方に触れていきましょう。

SOLIDの原則

S	: Single Responsibility	単一責任の原則
O	: Open-Closed	開放/閉鎖の原則
L	: LisKov Subsitution	リスコフの置換原則
I	: Interface Segregation	インターフェース分離の原則
D	: Dependency Inversion	依存性逆転の原則

これらは継承をするに当たって大切な考えになってきます。
具体的なコーディング等は割愛しますが、
プログラムの良し悪しを認識できるようにしましょう。

単一責任の原則

これはちよくちよく話してきたと思います。

単一責任の原則とはクラスが一つの振る舞い以外をするべきではないという考え方です。

皆さんがよくやる例としてはPlayerクラスなんかがなりがちですが、物理の処理もします。攻撃の処理もします。当たり判定もします。みたいなスクリプトはバグの温床になりがちです。

できる限り 1 クラスが 1 動作をするようにプログラムをしましょう。

開放/閉鎖の原則

この原則は、新しい要素の追加をする際に、既存のコードにテコ入れをしなくて良いように設計するべきという考えです。

例えば、急遽キャラクターにガードを追加したい。となったときに、今まで作ってあるキャラクターのUpdateの中に追加でプログラムを書かなくてはいけない、なんて事にならないようにしましょう。という考えです。

追加には開放的で、変更には閉鎖的な設計を目指すべきという意味です。

リスコフの置換原則

この原則は継承を行う場合は、親クラスを子クラスに入れ替えても同じふるまいをするようにしようという考えかたです。

継承については後で細かく解説しますが、
イメージとしては、

鳥クラスを継承した、ペンギンクラスが飛べなくてはならない
飛べないのであれば、鳥クラスを継承するべきではない
という考え方です。

インターフェース分離の原則

インターフェースについても後で細かく解説します。
継承の仲間だと思ってください。

この考えは簡単に言えば、いらない機能をつけるべきではないという考え方です。

例えば、歩いてジャンプして止まるスクリプトが別の場所にあり、それを流用して新しいスクリプトで歩く処理を書きました。
この際に、巻き込んでジャンプと止まるが出来るようになってはいけないという考え方です。

依存性逆転の原則

難しいのであまり気にしないでいいです。

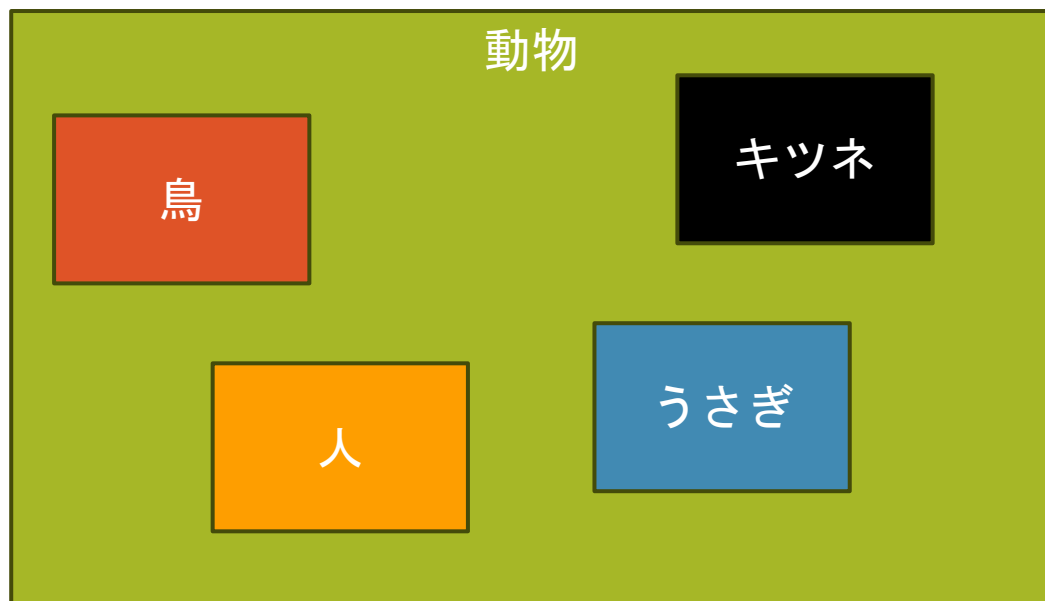
簡単にまとめると、ゲームマネージャーがプレイヤークラスを見るのは良くないしプレイヤークラスがゲームマネージャーを見るのも良くないという考え方です。

先程のインターフェースを嚙ませて、依存性を中和する事で、ゲームマネージャーがなくてもプレイヤーは動くし、プレイヤーがなくてもゲームマネージャーが動く状態を担保しようという考え方です。

継承について

雰囲気的な考え方ですが、グルーピングする事だと思ってください。

例えば



このような状態のときに継承を行います。

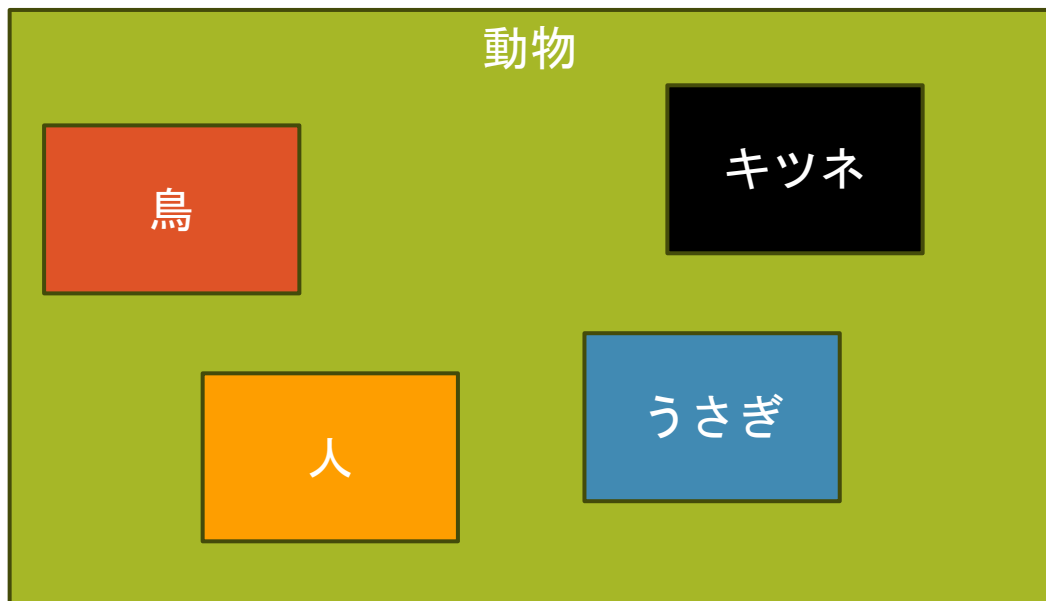
まず、動物クラスを作ります。
これを親クラスといいます。

次に動物クラスを継承して、
鳥、人、うさぎ、キツネのクラスを作ります。
これら一つ一つすべてを子クラスといいます。

継承のメリット

同じ振る舞いを使い回せる。

例えば



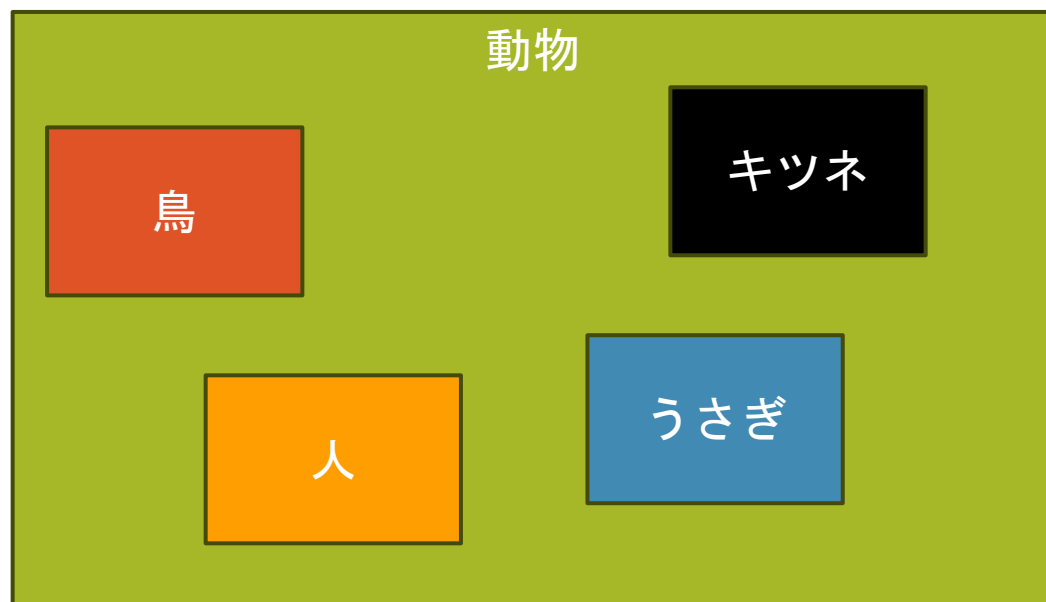
例えば、左の例のすべての動物が呼吸をします。その呼吸の処理を動物クラスに記入していれば、他のクラスでコーディングせずとも、呼吸の処理を使う事が出来ます。

これがないと、
鳥に呼吸書いて、
人に呼吸書いて、
うさぎに呼吸書いて、
キツネにも呼吸書いて、
みたいなあるだけ呼吸を書かなければ行けなくなります。

継承のメリット

特定の振る舞いを強制できる。

例えば



動物クラスであることさえわかれば、
実在しているものになるので、物理挙動を働かせたり、
食事しなかったらHPが減るようにしたり、
動物である事さえ確定していれば出来る処理を
一気に行う事が出来ます。

これが無いと、
鳥に〇〇をする
人に〇〇をする
キツネに〇〇をする
みたいなのをあるだけ書く必要が出てきます。

身近な継承

皆さんはUnityを触る時に当たり前のように継承の恩恵を受けています。

MonoBehaviourに見覚えがあると思います。

これはみなさんがスクリプトを作るときに、
`public class Player : MonoBehaviour`と勝手について来ています。

身近な継承 : MonoBehaviour

MonoBehaviourをつける事によって、いちいちオブジェクトにアタッチ出来るってコードを書かなくても良くなっています。

MonoBehaviourをつける事によって、StartやUpdateといった処理を強制的に呼び出してもらっていると思います。

こういった共通した処理を行わせるために継承を行います。

継承のグループとは

最初に継承はグループだと話したと思います。
イメージしやすいように動物に例えましたが、
本来は機能なのでグループをします。

- ・ 呼吸出来るやつ
- ・ 飛べるやつ
- ・ 歩けるやつ
- ・ クリックできるやつ、、、

こういったクラスとまではいかない機能の断片を
インターフェースとして定義します。

インターフェース

インターフェースとは2つの物事をつなげる方法や道具の事
プログラムに置いては関数の宣言だけを行うものです。

class, structなどと同じ部類で、

```
public interface I_Move  
{  
    public abstract void Move();  
}
```

みたいな書き方をします。

※インターフェースを定義するときはI_から始める事が多いです。

身近なインターフェース

UnityでI_PointerHandrerみたいなものをMonoBehaviourの隣に書いた事はありませんか？

このインターフェースをつけるとオブジェクトをクリックした際にOnPointerClick()関数が呼ばれるようになります。

このインターフェースはクリックされるというグループになり、このインターフェースを継承する事で、そのグループの仲間入りし、クリックを検知したら、OnPointerClick()を呼び出してくれるようになっています。

複数の継承

先ほど MonoBehaviour の隣に I_PointerHandler を書く といいましたが、継承は複数と同時に出来ます。

唯し、クラスの継承は 1 つでなければいけません (C#)
複数継承する場合は、コンマで区切って書きます。

```
public class Player : MonoBehaviour, I_PointerHandler, I_Singleton  
{  
  
}
```